

Programmazione Android

Tutorial: app "Blocco Note"

Prof. Marco Camurri

L'app di esempio "Blocco Note"

- Realizziamo un'app che consenta di salvare in modo *permanente* una nota testuale.



L'app di esempio

- La nota verrà salvata su file nella memoria del telefono, e quindi rimarrà memorizzata anche se si chiude l'app o si riavvia il telefono.
- Vedremo due alternative riguardo a "dove" salvare il file: cartella privata dell'app e cartelle pubblica
- In ogni caso, il salvataggio avverrà *in automatico* nel momento in cui l'utente sta per chiudere l'app (evento "onStop"). Non vi è quindi bisogno di un pulsante "salva"
- Il pulsante "condividi" consentirà di inviare il contenuto della nota tramite un'app di messaggistica a scelta tra quelle disponibili sul dispositivo (Esempio: GMail, WhatsApp, SMS, ecc...)

Argomenti

Tramite l'app di esempio affronteremo i seguenti argomenti:

- Assegnare i **pesi** ai componenti di un LinearLayout per gestire lo spazio in eccesso
- Salvare un **file nella cartella privata** dell'app
- Dichiarare i **permessi** e richiedere all'utente la **concessione** dei permessi
- Salvare un **file in una cartella pubblica** del dispositivo
- Avviare un'**activity esterna** alla nostra app (pulsante "condividi")

Creiamo l'interfaccia grafica

- Iniziamo creando un nuovo progetto (con il modello Empty Activity) e impostiamo i colori base della nostra app nel file *res/values/colors.xml*

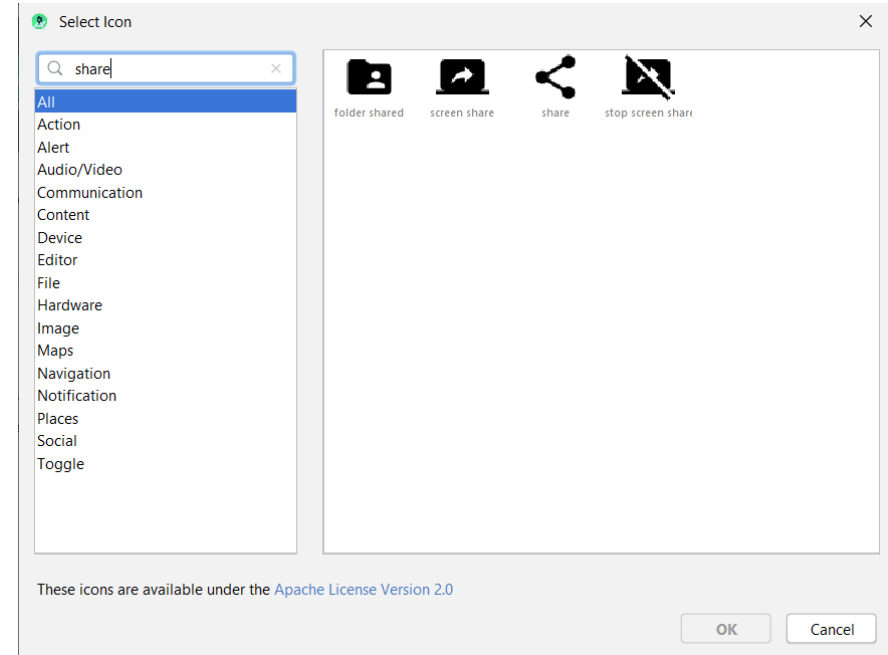
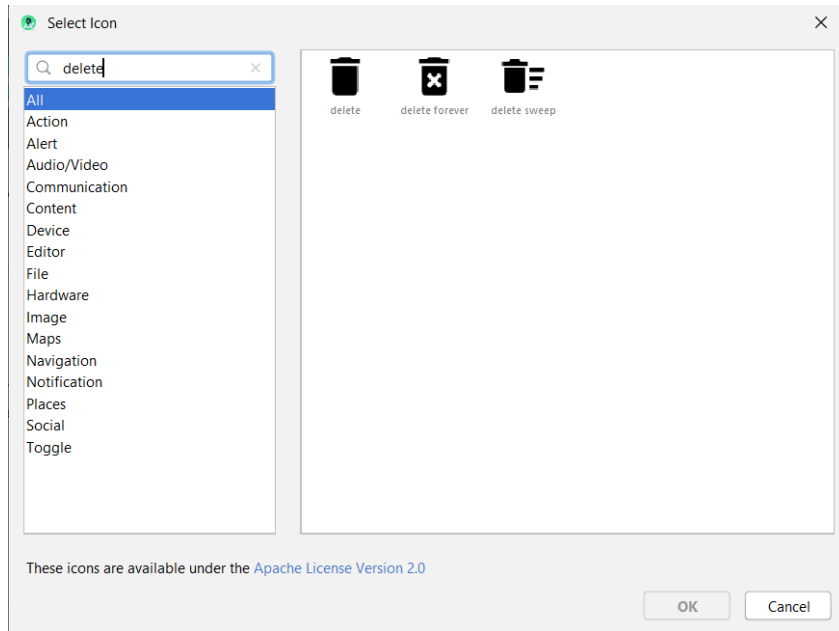
```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="colorPrimary">#EF6C00</color>
    <color name="colorPrimaryDark">#D84315</color>
    <color name="colorAccent">#03DAC5</color>

    <!-- .. lasciare gli altri colori eventualmente già presenti ... -->
</resources>
```

Nota: i primi due colori specificano rispettivamente il colore della barra del titolo e della barra delle notifiche

Creiamo l'interfaccia grafica

- Importiamo le immagini dei pulsanti come visto nella lezione scorsa (menu *File* | *New* | *Vector Asset* ...)



Creiamo l'interfaccia grafica

- Creiamo nella cartella *drawable* il file *sfondo_text_view.xml* con questo contenuto:

```
<?xml version="1.0" encoding="utf-8"?>
<shape
xmlns:android="http://schemas.android.com/apk/res/android"
    android:shape="rectangle">
    <corners android:radius="20dp" />
    <solid android:color="#99FFFFFF" />
</shape>
```

- In questo modo abbiamo creato uno sfondo bianco semitrasparente (#99FFFFFF) con bordi arrotondati, che useremo come background per la casella di testo

Creiamo l'interfaccia grafica

- Creiamo nella cartella *drawable* il file *sfondo_pulsante.xml* con questo contenuto:

```
<?xml version="1.0" encoding="utf-8"?>
<shape xmlns:android="http://schemas.android.com/apk/res/android"
    android:shape="oval">
    <solid android:color="#FFF176"/>
</shape>
```

- In questo modo abbiamo creato uno sfondo circolare che assegneremo ai due pulsanti in basso ("cancella" e "condividi")

Contenuto di *activity_main.xml*

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout

xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center_horizontal"
    android:background="#FFD54F"
    >

    <EditText
        android:id="@+id/casella"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Scrivi qui la tua nota"
        android:textSize="20dp"
        android:layout_weight="1"
        android:gravity="top"
        android:background="@drawable/sfondo_text_view"
        android:layout_margin="12dp"
        android:padding="20dp"/>
```

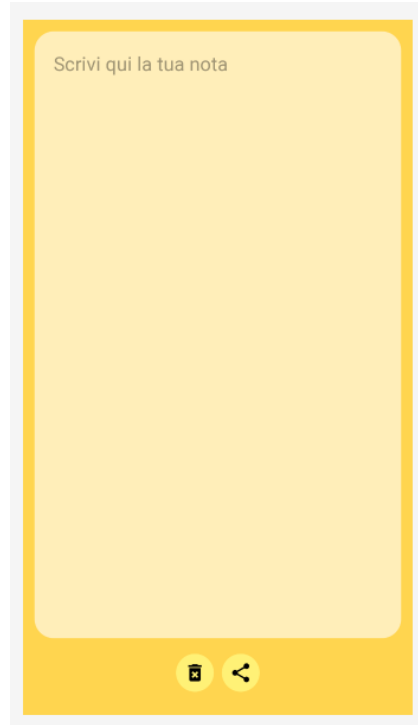
```
        <LinearLayout
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:orientation="horizontal"
            android:layout_marginBottom="20dp">
                <ImageButton
                    android:layout_width="40dp"
                    android:layout_height="40dp"
                    android:src="@drawable/ic_delete_forever_black_24dp"
                    android:background="@drawable/sfondo_pulsante"
                    android:layout_margin="4dp"/>
                <ImageButton
                    android:layout_width="40dp"
                    android:layout_height="40dp"
                    android:src="@drawable/ic_share_black_24dp"
                    android:background="@drawable/sfondo_pulsante"
                    android:layout_margin="4dp"/>
            </LinearLayout>
        </LinearLayout>
```

Osservazioni sul layout

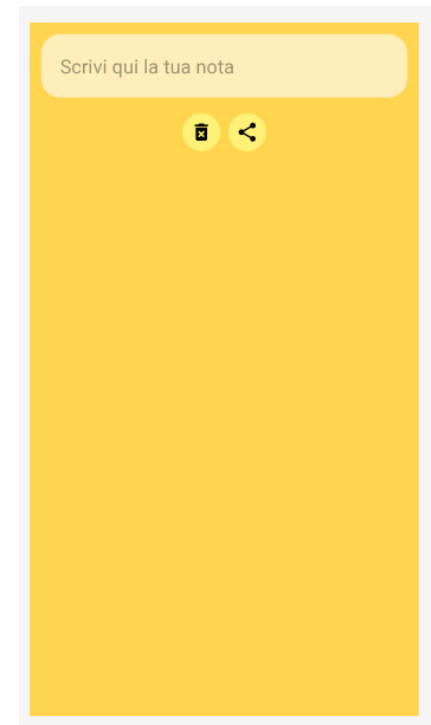
- Con il codice precedente abbiamo creato un `LinearLayout` più esterno, con orientamento verticale, al cui interno sono presenti una `TextView` (campo di testo editabile) e un secondo `LinearLayout` più interno (annidato).
- Il `LinearLayout` più interno è orientato in orizzontale e contiene i due pulsanti (`ImageButton`)
- L'attributo **gravity="center_horizontal"** nel `LinearLayout` esterno consente di centrare il blocco dei pulsanti (provare a rimuoverlo per vedere la differenza)
- L'attributo **gravity="top"** nella `TextView` indica la posizione del testo (provare a rimuoverlo per vedere la differenza)

La gestione dei pesi (*layout_weight*)

- L'attributo **layout_weight** può essere aggiunto a qualsiasi componente grafico all'interno di un `LinearLayout`, e consente di specificare come viene ripartito lo spazio in eccesso nella schermata



tutto lo spazio extra
è assegnato alla
`TextView`



tutto lo spazio extra
è lasciato "vuoto"

La gestione dei pesi (*layout_weight*)

- Lo spazio in eccesso viene distribuito tra i componenti in proporzione al loro peso
- il peso di default per i componenti è *zero*
- impostando **layout_weight="1"** sulla TextView, quindi, tutto lo spazio in eccesso della verrà attribuito ad essa
- In altre parole: impostando il peso 1 sulla TextView e peso 0 su tutti gli altri componenti, otterremo che la TextView si espande in altezza fino a occupare tutto lo spazio residuo della schermata.

La gestione dei pesi (*layout_weight*)

Volendo distribuire lo spazio in eccesso in questo modo:

- 2/3 dello spazio in eccesso alla TextView
- 1/3 dello spazio in eccesso al gruppo di pulsanti

sarebbe sufficiente assegnare peso 2 alla TextView e peso 1 alla LinearLayout interno (provare per vedere il risultato).

Nota: Android calcola **la somma dei pesi** e assegna lo spazio extra in proporzione a tale somma. Quindi otterremmo lo stesso risultato anche assegnando rispettivamente i pesi 200 e 100. Ciò che conta è il rapporto tra il peso di un componente e la somma totale dei pesi.

Salvataggio nella cartella privata

Per gestire il salvataggio su file dobbiamo effettuare due modifiche al file MainActivity.java:

1) effettueremo l'override del metodo onStop. Tale metodo viene richiamato automaticamente da Android quando la nostra app sta per essere chiusa. In questo metodo otterremo il percorso della cartella privata (l'unica in cui possiamo scrivere senza ulteriori permessi) e salveremo qui un file chiamato *nota.txt*

2) modificheremo il metodo onCreate per far sì che, all'apertura dell'app, il contenuto del file *nota.txt* (se esiste) sia caricato nella TextView.

Salvataggio nella cartella privata

1) Implementazione del metodo onStop:

```
@Override
protected void onStop() {
    super.onStop();
    TextView campo = findViewById(R.id.casella);
    String stringa = campo.getText().toString(); // necessario il toString
    File cartella = getFilesDir();
    Log.d("MioTag", "Salvo nella cartella " + cartella.getAbsolutePath() );
    try {
        FileWriter fw = new FileWriter(cartella.getAbsolutePath()+"/nota.txt" );
        fw.write(stringa);
        fw.close();
        Toast.makeText(this, "Nota salvata", Toast.LENGTH_SHORT).show();
    } catch (IOException e) {
        Toast.makeText(this, "Errore", Toast.LENGTH_LONG).show();
    }
}
```

Spiegazione del codice

- il metodo **getFilesDir** della classe Activity ci restituisce un oggetto di tipo *File* che rappresenta la *cartella privata* della nostra app
- Questa è l'unica cartella in cui la nostra app può leggere e scrivere file senza permessi aggiuntivi
- E' importante usare SEMPRE questo metodo per ottenere il percorso della cartella privata, in quanto il percorso esatto può variare da dispositivo a dispositivo (*non è possibile "cablare" il percorso nel codice*)

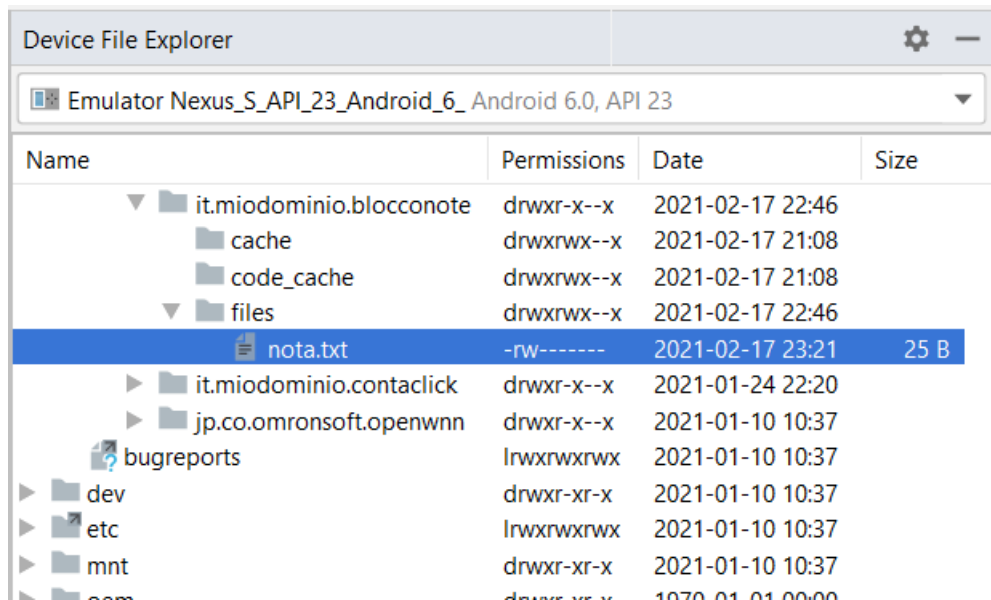
Salvataggio nella cartella privata

2) Modifica al metodo **onCreate**:

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    File cartella = getFilesDir();
    try {
        String nomeFile = cartella.getAbsolutePath() + "/" + "nota.txt";
        File f = new File(nomeFile);
        if ( f.exists() ) {
            BufferedReader reader = new BufferedReader(new FileReader(f));
            String testoNota = "", riga;
            while ((riga = reader.readLine()) != null) {
                testoNota += riga;
            }
            TextView campo = findViewById(R.id.casella);
            campo.setText(testoNota);
        }
    } catch (IOException e) {
        Toast.makeText(this, "Errore durante la lettura", Toast.LENGTH_LONG).show();
    }
}
```

Salvataggio nella cartella privata

- Da Android Studio è possibile verificare la presenza del file tramite il "device explorer" (normalmente attivabile tramite la linguetta in basso a destra dell'IDE)



Percorso:

/data/user/0/
it.miodominio.bloconote/files/
nota.txt

Notare i permessi: il file è accessibile solo dalla nostra app.

NOTA BENE: il percorso esatto può variare da dispositivo a dispositivo.

Gestione del pulsante "condividi"

- Per gestire il pulsante "condividi" aggiungiamo il seguente metodo nel file *MainActivity.java*

```
public void condividi(View v) {  
    TextView campo = findViewById(R.id.casella);  
    String msg = "Ecco la mia nota: \n" + campo.getText();  
    Intent intent = new Intent();  
    intent.setAction(Intent.ACTION_SEND);  
    intent.putExtra(Intent.EXTRA_TEXT, msg);  
    intent.setType("text/plain");  
    startActivity( intent );  
}
```

e impostiamo opportunamente l'attributo onClick del pulsante nel file *activity_main.xml*

Spiegazione del codice

- Con il codice precedente abbiamo creato un Intent *implicito*
- Un intent implicito rappresenta l'intenzione di avviare un'activity di cui non conosciamo il *nome esatto* ma solo la generica *azione* (che vorremmo svolgere con tale activity)
- In particolare, specificando l'azione **ACTION_SEND** abbiamo richiesto al sistema operativo di individuare, tra tutte le app installate, quelle in grado di "inviare messaggi"
- con il metodo **putExtra** possiamo passare dei dati all'Activity che verrà aperta, sottoforma di coppie (chiave,valore). In questo modo l'app di messaggistica prescelta potrà aprirsi con il messaggio già "composto" e pronto per essere inviato.

Salvataggio nella cartella pubblica

Per salvare il file in una cartella "pubblica" del telefono (cioè in una cartella accessibile dall'utente e dalle altre app) occorre effettuare due operazioni:

- 1) **Dichiarare** il permesso di lettura e scrittura nel file Manifest.xml
- 2) Richiedere all'utente la **concessione** del permesso

Dichiarazione dei permessi

- Aggiungere nel file *Manifest.xml* le due righe evidenziate:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="it.miodominio.bloconote">
    <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>
    <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/AppTheme">
        <activity android:name=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

Richiesta dei permessi

- Modificare il file MainActivity.java come segue:

```
package it.miodominio.bloconote;

import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.app.ActivityCompat;
import androidx.core.content.ContextCompat;
import android.Manifest;
import android.content.*;
import android.os.Bundle;
import android.os.Environment;
import android.util.Log;
import android.view.View;
import android.widget.*;
import java.io.*;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        if ( ContextCompat.checkSelfPermission(this,Manifest.permission.WRITE_EXTERNAL_STORAGE)
            != PackageManager.PERMISSION_GRANTED ) {
            ActivityCompat.requestPermissions(this, new String[] {Manifest.permission.WRITE_EXTERNAL_STORAGE}, 0 );
        }
        else {
            leggiFile();
        }
    }
}
```

Richiesta dei permessi

```
@Override
protected void onStop() {
    super.onStop();
    TextView campo = findViewById(R.id.casella);
    String stringa = campo.getText().toString(); // necessario toString poichè getText restituisce Charsequence e non String
    //File cartella = getFilesDir();
    File cartella = getExternalFilesDir("Note");
    Log.d("MioTag", "Salvo nella cartella " + cartella.getAbsolutePath() );
    try {
        FileWriter fw = new FileWriter( cartella.getAbsolutePath() + "/" + "nota.txt" );
        fw.write(stringa);
        fw.close();
        Toast.makeText(this, "Nota salvata", Toast.LENGTH_SHORT).show();
    } catch (IOException e) {
        Toast.makeText(this, "Errore durante il salvataggio", Toast.LENGTH_LONG).show();
        e.printStackTrace();
    }
}
```


Richiesta dei permessi

```
@Override
public void onRequestPermissionsResult(int requestCode, @NonNull String[] permissions, @NonNull int[] grantResults) {
    super.onRequestPermissionsResult(requestCode, permissions, grantResults);
    if (grantResults[0] == PackageManager.PERMISSION_GRANTED) {
        Toast.makeText(this, "Grazie di aver concesso il permesso!", Toast.LENGTH_SHORT).show();
        leggiFile();
    }
    else {
        Toast.makeText(this, "Senza il permesso non posso fare nulla", Toast.LENGTH_SHORT).show();
        finish();
    }
}

public void leggiFile() {
    //File cartella = getFilesDir();
    File cartella = getExternalFilesDir("Note");
    try {
        String nomeFile = cartella.getAbsolutePath() + "/" + "nota.txt";
        File f = new File(nomeFile);
        if ( f.exists() ) {
            BufferedReader reader = new BufferedReader(new FileReader(f));
            String testoNota = "", riga;
            while ((riga = reader.readLine()) != null) {
                testoNota += riga;
            }
            TextView campo = findViewById(R.id.casella);
            campo.setText(testoNota);
        }
    } catch (IOException e) {
        Toast.makeText(this, "Errore durante la lettura", Toast.LENGTH_LONG).show();
    }
}

} // fine classe
```